Slipshow: chill-coding with OCaml

Paul-Elliot

July 13, 2026

1 Introduction

Slipshow is a presentation tool designed to innovate beyond the traditional "deck of slides" paradigm. For instance, it introduces "slips": bottomless slides that you scroll through to reveal content. Another unique feature is the ability to record hand drawings, edit them, and replay them live during a presentation.

2 Growing the project

Slipshow began as a single JavaScript file, embeddable in specific HTML pages to turn them into presentations. This file quickly became unmaintainable; basic features were buggy, and fixes repeatedly broke other parts of the codebase.

Today, it is a substantial project featuring a runtime, a compiler, an LSP server with a live preview window, and a dedicated timeline editor for hand drawings, backed by an NLNet grant. How did the project finally scale from its unmaintainable origins?

2.1 Chill-coding

Today, the common cause for enlarged personal projects is "vibe-coding": relying entirely on AI to generate code without reading it. Freed from the friction of interacting with the code, the cognitive load is lowered.

Given the name and the good vibes I get while programming, I initially thought I was vibe-coding. However, two issues arose: first, I do not use AI to generate code. Second, vibe-coding means losing touch with your software's internals, making it difficult to trust or safely distribute without exhaustive testing.

More than vibes, I feel *chill* while coding: chill during massive refactors, chill when adding features, and chill for feeling safe from introducing bugs or accidental complexity. This is *chill-coding*: the act of programming without fear.

Chill-coding requires either exceptional skill or support from exceptionally powerful tools. I'm in the latter category. After years of friction with the JavaScript prototype, I moved to OCaml for every part of Slipshow. Backed by intelligent compiler errors, excellent editor tooling, a robust library ecosystem, and a helpful community, I have been able to chill-code ever since.

3 An experience report

Writing OCaml has been a joy from the start. It feels safe and lets your creativity wander, without friction. Through snippets of Slipshow's Git history, I want to share this excitement for the language, its tooling, and its ecosystem.

3.1 A runtime in JavaScript

Slipshow began as a presentation engine written in JavaScript. Here is a snapshot of historical commits all attempting to fix a *single* basic feature: navigating backward. It was endless.

```
436bb65  implemented engine.previousSlip
d0cfc69  corrected "not coming beginning on previous" bug
61e6cb7  fix "future slip, delay not 0 again when previousing"
3af56a3  corrected horribug previous
bcd8323  better previous:  use delay from previous state
28372cd  two bugs in previous and annotations
2437301  correcting bug with "previous" and "resizeObserver"
```

3.2 OCaml for the compiler

Commit `0fa489e` introduced the first lines of OCaml to the project, adding a compiler that translates Markdown files into standalone HTML presentations.

OCaml is known to excel at compiler implementation, making the core of the compilation fairly straightforward. But through its ecosystem, it also excels at all the surrounding tasks, such as CLIs with `cmdliner`, error reporting with `grace`, and Markdown parsing with `cmarkit`.

OCaml supports both normal and extensible variants. `cmarkit`'s AST using the latter, I was able to only implement my new custom blocks, and rely on `cmarkit`'s high-quality operators for the core of the AST.

Note that extensible variants used plainly make you lose match exhaustiveness checks. In my quest to being protected by compiler errors, commit `5713489` restored the chill by wrapping extensions in a standard variant:

```
type s_block =
| Slip of Block.t attributed node
| SlipScript of Block.Code_block.t attributed node
| [...]

type Cmarkit.Block.t += S_block of s_block
```

```
5713489  "Make it more suitable for match exhaustiveness check"
```

3.3 The runtime, rewritten in OCaml

For a while, the JavaScript runtime and the OCaml compiler coexisted. Because cross-language communication and the runtime itself were fragile, I decided to rewrite the runtime entirely in OCaml, making the project OCaml only.

The effect is only positive, as expressed by this commit which solved the backward navigation bugs that plagued the JavaScript era:

`32d0917` *"BOOOOOOOOOOOOOOMM! (Undo monad, they work)"*

Beyond the undo monad, tools like `dune`, `js_of_ocaml`, and `ppx_blob` made managing Slipshow's non-trivial build process seamless: the runtime is compiled to bytecode, translated to JavaScript, and directly embedded into the compiler binary.

```
(executable
  (name engine)
  (modes js))

(executable
  (name compiler)
  (preprocess (pps ppx_blob))  ; Use with [%blob "engine.bc.js"]
  (preprocessor_deps engine.bc.js))
```

3.4 Record and replay drawings

A recent game-changing addition is the ability to record, edit, and replay hand drawings during presentations.

Building the timeline editor, inspired by video editors, was surprisingly joyful despite the inherent UI challenges. This feeling is captured perfectly in a commit:

`3ac99e3` *"Wo so cool lwd"*

The joy stems from using high-quality libraries from the ecosystem that make a hard task easy. `Lwd` is a reactive programming library that automatically updates the UI when underlying data changes. It is a powerful technique perfectly suited to my use-case. For instance, filtering drawing coordinates based on the `elapsed_time` becomes incredibly clean and declarative using custom binding operators:

```
let filter_path elapsed_time path =
  let$* path = path in      (* Both inputs are reactive values *)
  let$ elapsed_time = elapsed_time in
  List.filter (fun (_, t) -> t < elapsed_time) path
```

4 Conclusion

Slipshow's growth and the fun of chill-coding it are entirely due to OCaml's exceptional language design, compiler, ecosystem, and community. I hope this glimpse into its development history inspires others to experience this style of programming.